

Assignment 1

In22-S5-EN3160 - Image Processing and Machine Vision

Electronic and Telecommunication Engineering

University of Moratuwa



Rajakaruna R.W.M.D.S. - 220498D

Github Link : **Assignment 1**

August 12, 2025

Question 1

```
1 #Given Intensity Transformation Function Application for an image of Emma Watsons
2 in_intensity=np.arange(256, dtype=np.uint8)
3 #initializing the output Intensity array
4 out_intensity=np.zeros(256, dtype=np.uint8)
5 # Define the transformation curve
6 out_intensity[:51]=1*in_intensity[:51]
7 out_intensity[51:151]=1.55*(in_intensity[51:151]-50)+100
8 out_intensity[151:256]=1*(in_intensity[151:256])
```

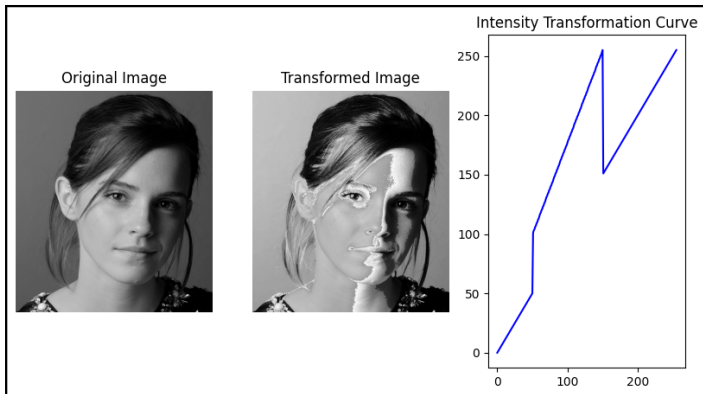


Figure 1: Intensity Transformation

Here in the transformation function uplift the intensities of the pixels which initially has the intensity levels from 50 to 150 (mid range or the gray range pixel intensities) and other (High Intensity and Low Intensity) intensities remains unchanged

Question 2

```
1 #Accentuation of White Matter
2 in_intensity=np.arange(256, dtype=np.uint8)
3 out_intensity=np.zeros(256, dtype=np.uint8)
4 threshold=180
5 # Define the transformation curve
6 out_intensity[:threshold]=0
7 out_intensity[threshold+1:256]=1*(in_intensity[threshold+1:256])
```

```
1 #Accentuation of Gray Matter
2 in_intensity=np.arange(256, dtype=np.uint8)
3 out_intensity=np.zeros(256, dtype=np.uint8)
4 upbound=200
5 lowbound=80
6 # Define the transformation curve
7 out_intensity[:lowbound]=0
8 out_intensity[lowbound+1:upbound]=1*(in_intensity[lowbound+1:upbound])
9 out_intensity[upbound+1:256]=0
```

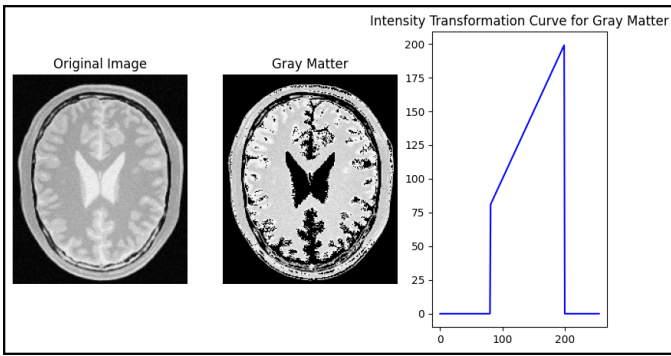


Figure 2: Accentuated gray matter

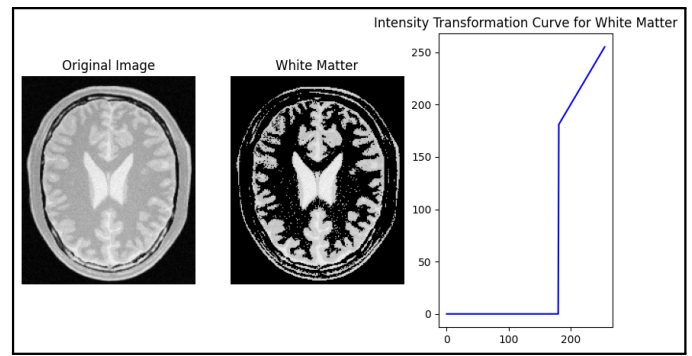


Figure 3: Accentuated white matter

To enhance white matter in the image, all pixel intensities below 180 were set to zero, while intensities above this threshold were left unchanged. Similarly, to enhance gray matter, pixel intensities within the threshold range were preserved, and all other intensities were set to zero, resulting in a darker background.

Question 3

```

1 #Gamma value is selected as 0.72 whcih gives a fair output
2 L,a,b = cv.split(image_LAB)
3 gamma=0.72
4 out_intensity= (((in_intensity/255)**(gamma))*255).astype(np.uint8)
5 newL=cv.LUT(L, out_intensity)
6 GammaCorrectedImg = cv.merge([newL, a, b])

```

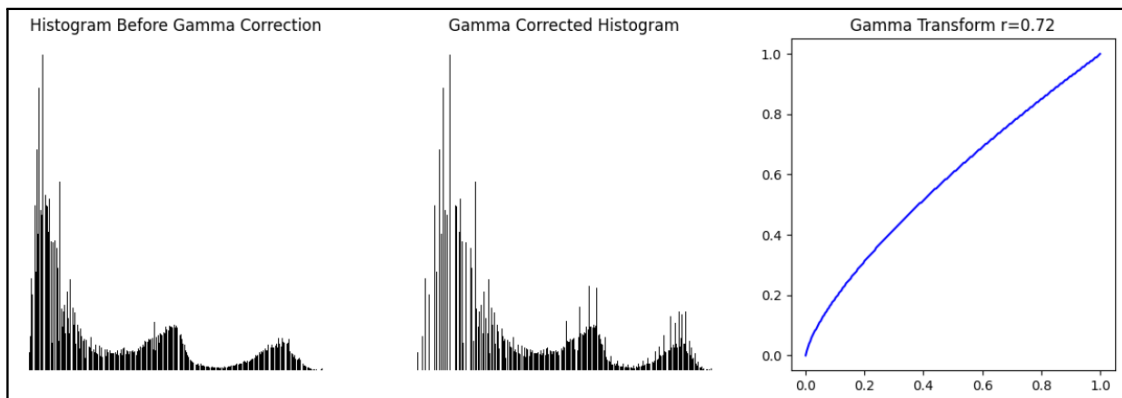


Figure 4: Histograms of L plane and Gamma function

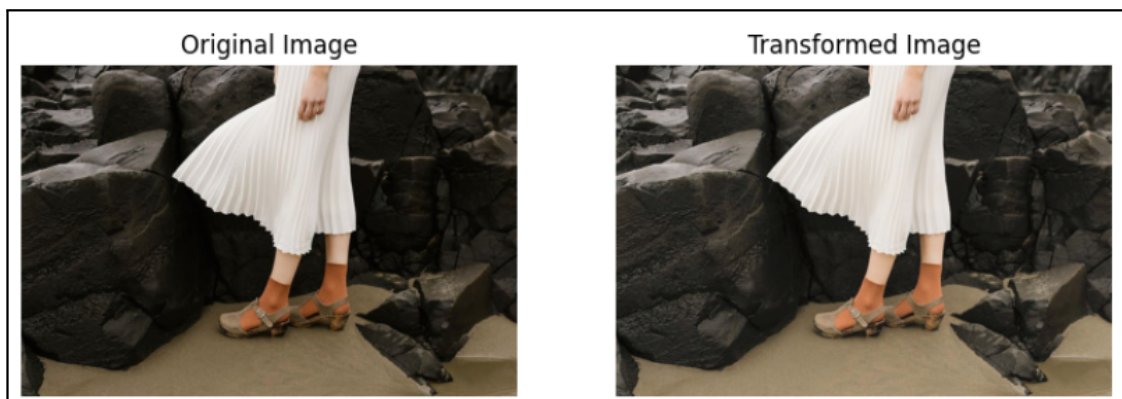


Figure 5: Original and Gamma Corrected Images

Gamma correction is applied to the L channel of the image in the LAB color space. The transformation was implemented using a lookup table. Histograms of the corrected image shows increased noise and this is a non linear operation.

Question 4



Figure 6: HSV Planes of the image



Figure 7: Intensity Transformation and vibrance-enhanced image

```
1 saturation_img=spider_img_HSV[:, :, 1]
2 copy_img=spider_img_HSV.copy()
3 zigma=70
4 a=0.6 # a E [0,1]
5 edited_saturation_img = np.minimum(saturation_img + a *128* np.exp(-((saturation_img-128)**2)/(2*zigma**2)),
  ↳ 255).astype(np.uint8)
```

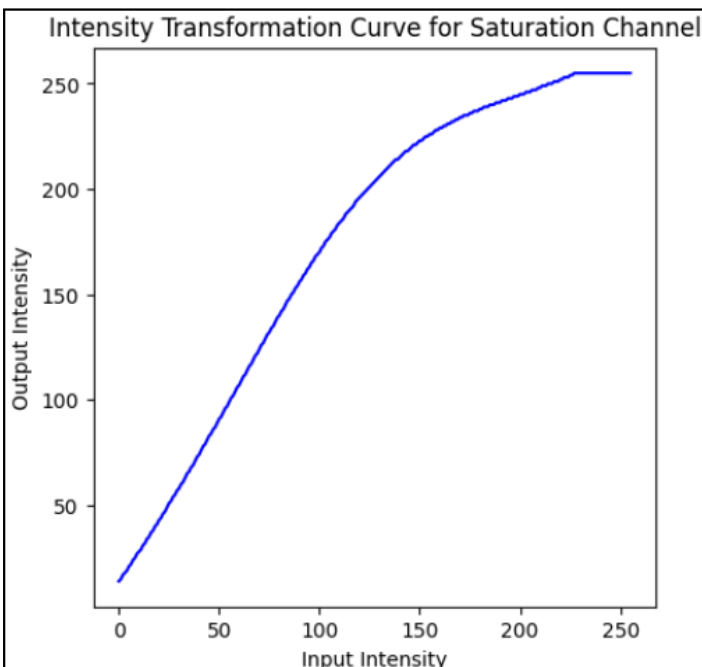


Figure 8: Intensity Transformation Curve

Image splitted into Hue, Saturation and Value planes and performed the intensity transformation for the saturation plane and using the given function. Set the sigma to 70 and found that $a = 0.6$ gives better results after transformation. This amplifies the colorful areas while applying less effect on the other areas.

Question 5

```
1 def equalization(flatArray):
2     frequency = np.bincount(flatArray, minlength=256)
```

```

3     cdf = np.cumsum(frequency)
4     cdf_normalized = np.round((255 * cdf) / cdf[-1]).astype(np.uint8) #function
5     return cdf_normalized, frequency
6 flatArray = shells_img.ravel()
7 cdf_normalized, frequency = equalization(flatArray)
8 equalized_flat = cdf_normalized[flatArray] # Apply mapping
9 equalized_img = equalized_flat.reshape(shells_img.shape)

```

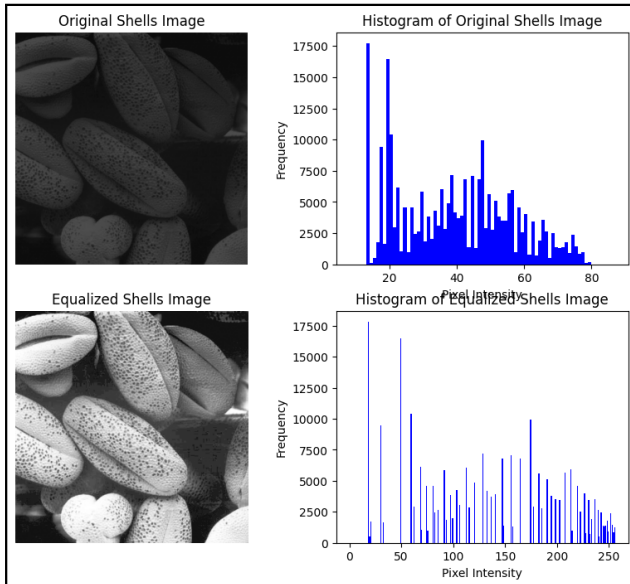


Figure 9: Histogram Equalization

The histogram of the image is more spreaded after the equalization and have introduces some noises. Due to the achieving high intensities the color of the image is brighter now.

Question 6

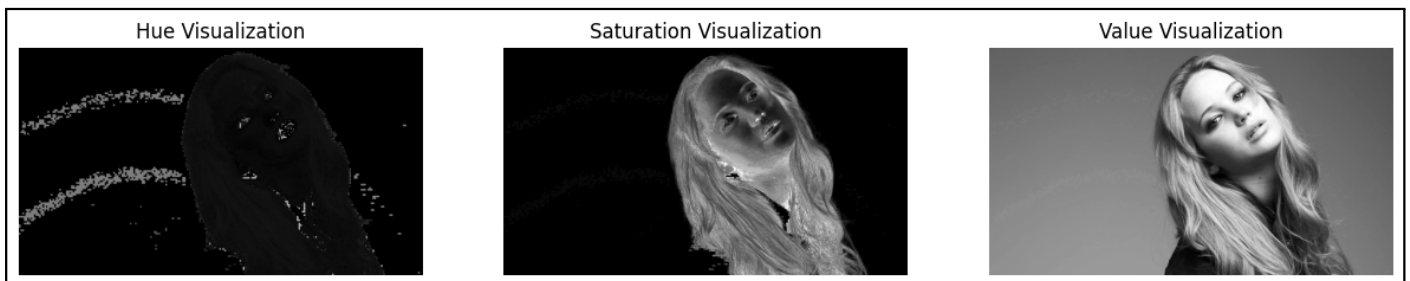


Figure 10: Extracted HSV Planes

```

1 s_channel = jeniffer_img_HSV[:, :, 1] # Saturation channel is choosed
2 black_th = 12
3 white_th = 12
4 # Background mask
5 girl_mask = np.where(s_channel < black_th, 0, 255).astype(np.uint8)
6 # Girl mask
7 backgroud_mask = np.where(s_channel > white_th, 0, 255).astype(np.uint8)

```

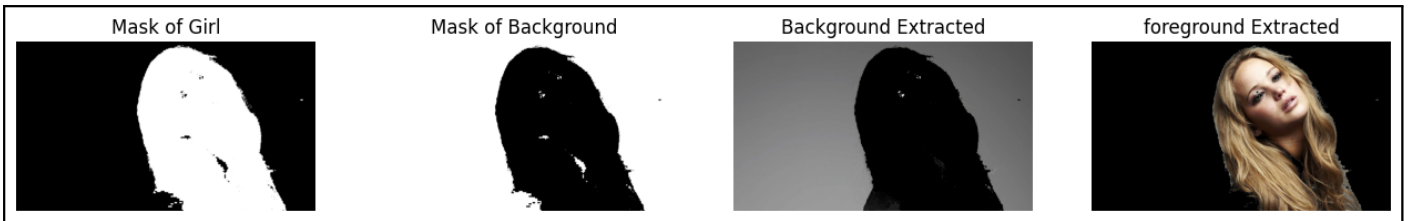


Figure 11: Extracted Masks, Background and the foreground

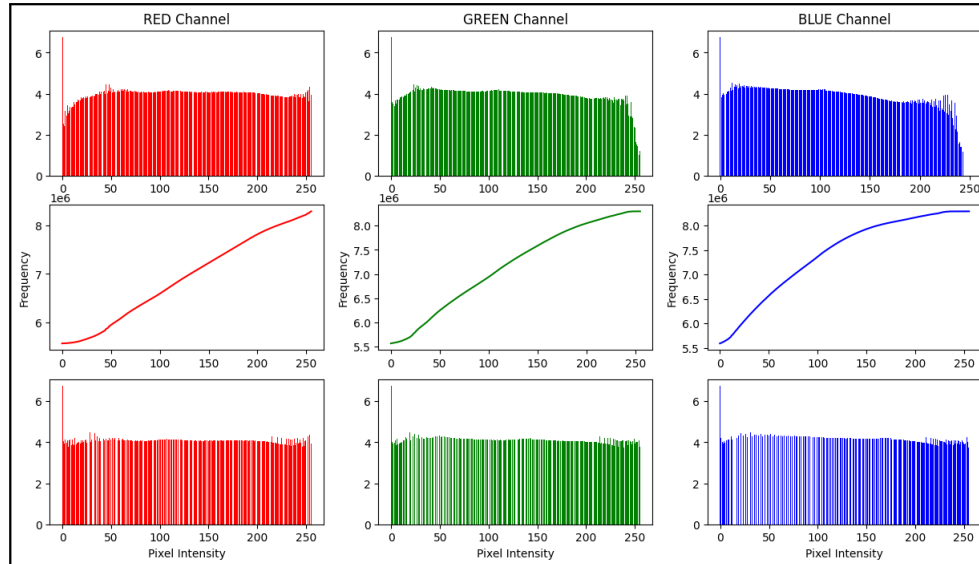
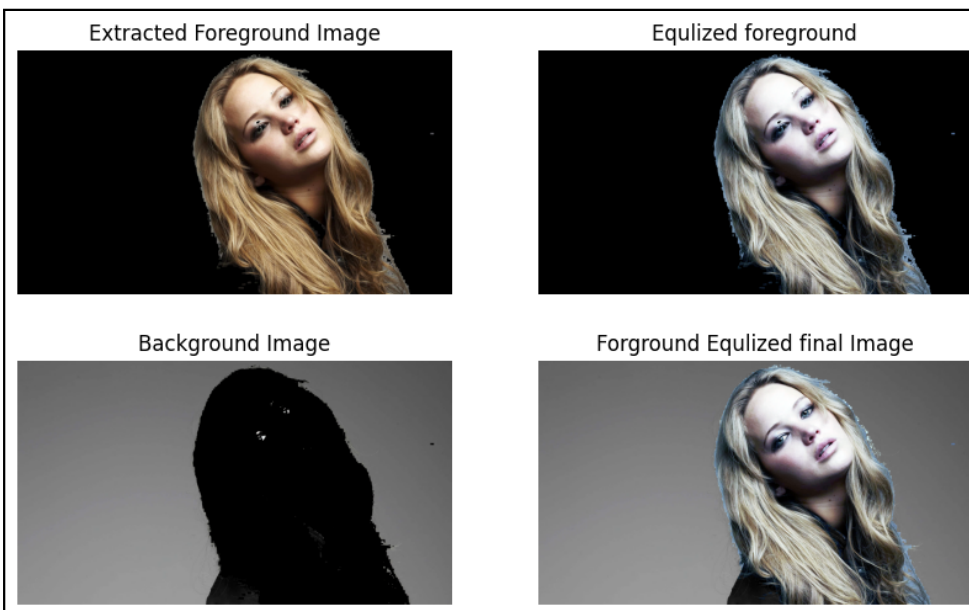


Figure 12: Original and equalized histograms and the CDFs of each RGB planes



Here only the foreground is extracted from the image and have performed the histogram equalization only for the foreground and for all RGB planes separately. In visualization I have used Log scale to have a much more detailed visualization. So in the final image the background remains unchanged but the foreground is changed.

Figure 13

Question 7

```

1 def sobelFilter(image,kernal):
2     if len(image.shape) == 3:
3         image = cv.cvtColor(image, cv.COLOR_BGR2GRAY)
4     img_padded = cv.copyMakeBorder(image,top=1, bottom=1, left=1, right=1,borderType=cv.BORDER_CONSTANT,
5         value=0)
6     rows, cols = image.shape

```



```

6  new_img = np.zeros((rows, cols), dtype=np.float64)
7  for i in range(rows):
8      for j in range(cols):
9          block=img_padded[i:i+3,j:j+3]
10         new_img[i,j]=np.sum(block*kernal)
11  return new_img

```

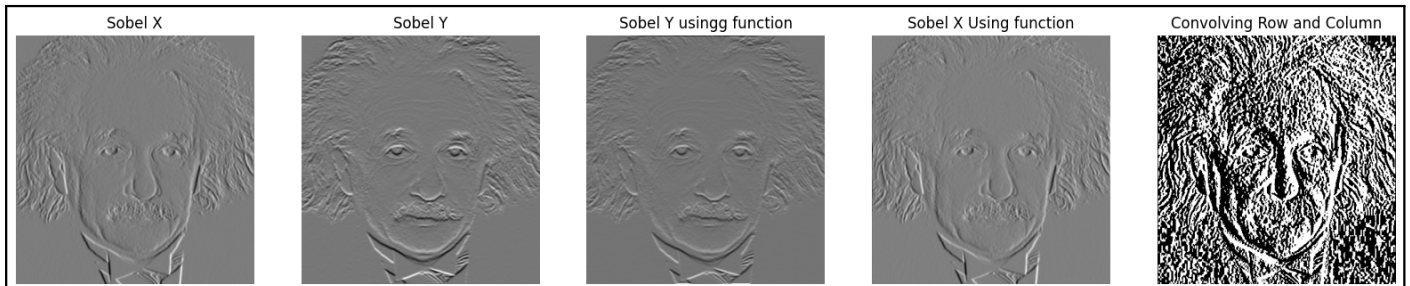


Figure 14: Sobel operation usage for Edge detection

Question 8

```

1  def zoom_image(img, method, factor): #Need to edit here  uefweififuwifiui
2      if(10<factor or factor<0):return v
3      if (method == "nearest"):
4          interpolation=cv.INTER_NEAREST
5      else:
6          interpolation=cv.INTER_LINEAR
7      (h,w,_) = img.shape
8      new_size = (int( w * factor), int(h * factor))
9      return cv.resize(img, new_size, interpolation=interpolation)

```



Figure 15: Zoomed images using Nearest Neighbor and Bilinear Interpolation

Question 9

```

1  mask = np.zeros(daisyImg.shape[:2], np.uint8)
2  backgroundModel = np.zeros((1, 65), np.float64)
3  foregroundModel = np.zeros((1, 65), np.float64)
4  x, y, w, h = 50, 100, 520, 450
5  rectangle = (x,y,w,h)
6  cv.grabCut(daisyImg, mask, rectangle,backgroundModel, foregroundModel,3, cv.GC_INIT_WITH_RECT)
7  mask2 = np.where((mask == 2)|(mask == 0), 0, 1).astype('uint8')

```

```

8 image_segmented = daisyImg * mask2[:, :, np.newaxis]
9 background = cv.subtract(daisyImg, image_segmented)
10 blurredBack=cv.GaussianBlur(background,(29, 29), 3)

```

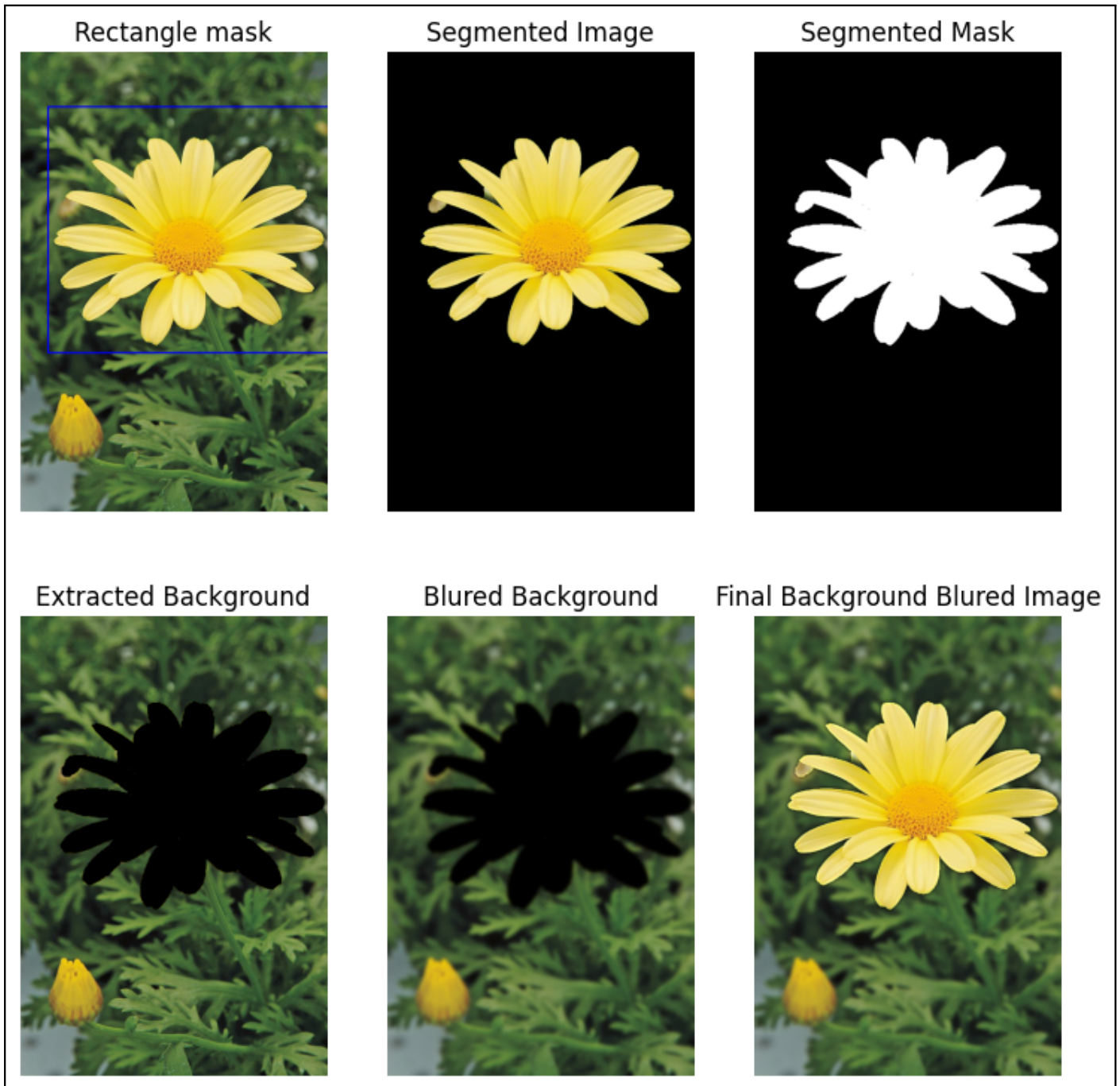


Figure 16: Background Blurring

The reason to being dark the background, just beyond the edge of the flower of enhanced image is when we apply the gaussian filter to blur the background, that filter works as an averaging filter. Then it has the effect on the black pixels of the Extracted Background Image. So it brings slight darkness for the pixels near the black mask edge.